THE UNIVERSITY OF AZAD JAMMU AND KASHMIR

Department of Software Engineering

DIGITAL IMAGE PROCESSING LAB

Lab Task # 01

Python Setup, Fundamentals, and Imports

NAME	**Syeda Khadija Hassan**	ROLL NO	**2024-SE-09**
DEPARTMENT	Software Engineering	SESSION	2024-28 (SE)
SEMESTER	Fifth	INSTRUCTOR	Engr. Fahad Nisar
ASSIGNED	October 16, 2026	DUE	October 23, 2026, 11:59 PM

OBJECTIVE

Install Python and VS Code, practice basic Python syntax (conditions, loops, functions and collections), install the introductory packages NumPy, Pillow and Matplotlib, and verify the required import statements.

LAB AT A GLANCE

Step	Task	Deliverable
1 – 3	Install Python, verify pip, set up VS Code	Version checks, interpreter
4	Run the first Python program	hello.py
5	Six basic Python exercises	basics.py
6	Install NumPy, Pillow, Matplotlib	pip show output
8	Write and verify import statements	lab01_imports.py

1 STEP 1 Install Python

Python 3 was downloaded from python.org/downloads and installed on Windows with the option “Add Python to PATH” enabled. The Command Prompt was then closed and reopened so the new PATH took effect.

> **TIP**
> If the PATH option is missed during installation, the python command will not be recognised in Command Prompt. Re-run the installer and enable it.

2 STEP 2 Verify Python and pip

A new Command Prompt window was opened and the following commands were run to confirm that both Python and pip are installed.

Command Prompt

```
dip-lab-01>python --version
dip-lab-01>python -m pip --version
```

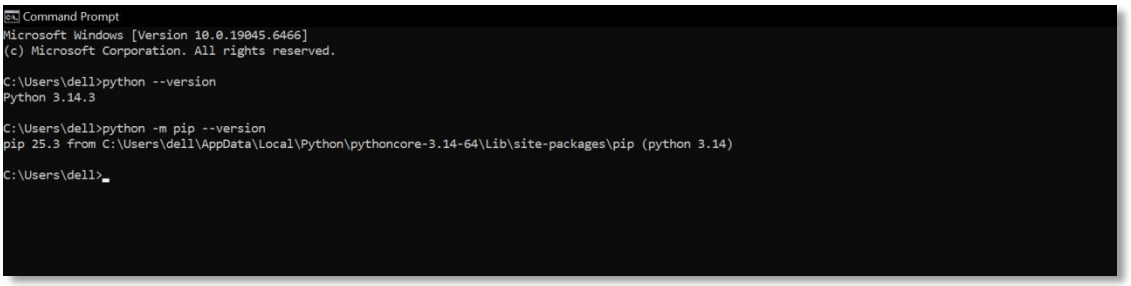


Figure 1 · Python and pip versions

3 STEP 3 Install VS Code and Select Python

VS Code was installed from code.visualstudio.com. The “Python” extension by Microsoft was installed from the Extensions panel (Ctrl + Shift + X), and the interpreter was chosen using Ctrl + Shift + P → Python: Select Interpreter. The project folder dip-lab-01 holds all lab files.

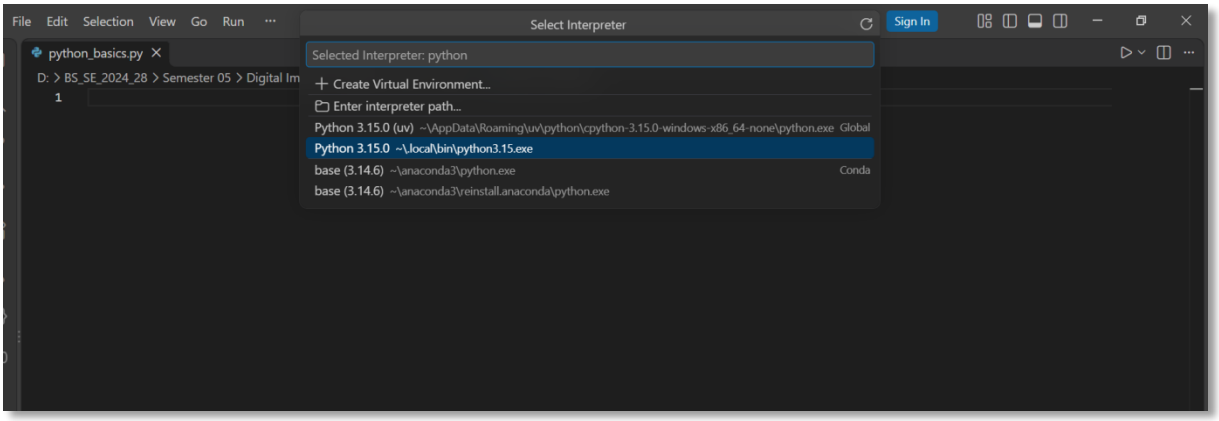


Figure 2 · Selected VS Code interpreter

4 STEP 4 Run Your First Python Program

hello.py was created in VS Code and executed from Command Prompt inside the dip-lab-01 folder.

hello.py

```
# My first Python program
print('Hello, Digital Image Processing!')
```

Command Prompt

```
dip-lab-01>python hello.py
Hello, Digital Image Processing!
```

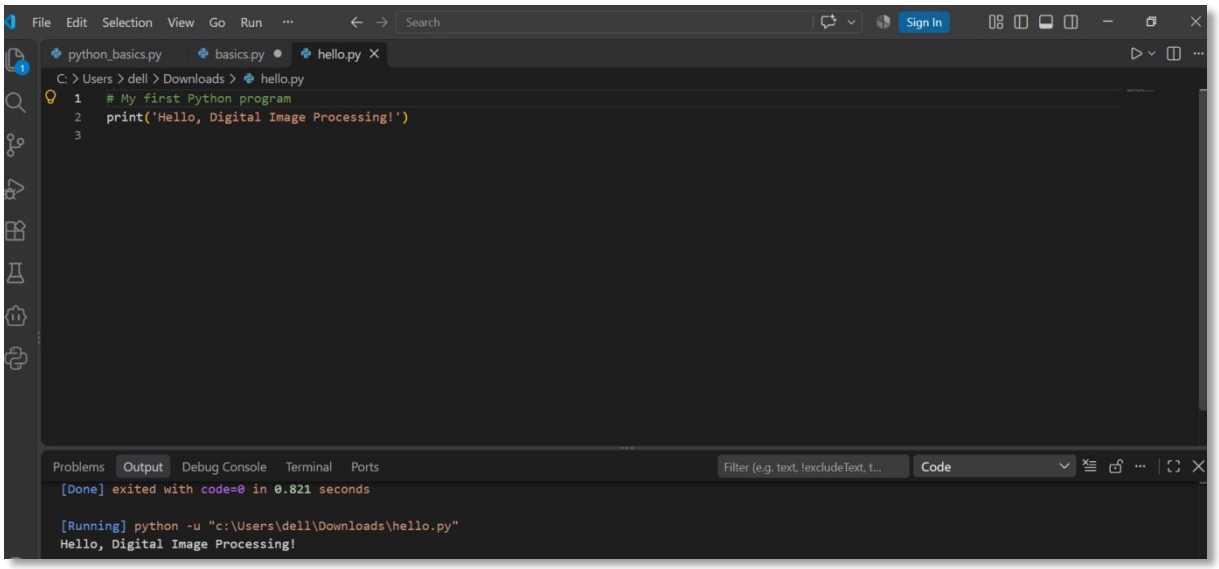


Figure 3 · hello.py and its output

5 STEP 5 Basic Python Exercises

All six exercises are written in basics.py with a labelled comment for each section, using four-space indentation. The file was run with:

Command Prompt

```
dip-lab-01>python basics.py
```

EXERCISE 1 Variables and Data Types

Four variables of different types are created and printed together with their types. The total number of pixels is calculated as width * height.

basics.py · Exercise 1

```python
# Exercise 1: Variables and Data Types
print("=== Exercise 1: Variables and Data Types ===")
width = 640
height = 480
image_name = "sample.png"
is_color = True

print("width =", width, "| type:", type(width))
print("height =", height, "| type:", type(height))
print("image_name =", image_name, "| type:", type(image_name))
print("is_color =", is_color, "| type:", type(is_color))

total_pixels = width * height
print("Total number of pixels:", total_pixels)  # Expected: 307200
```

Output

```
=== Exercise 1: Variables and Data Types ===
width = 640 | type: <class 'int'>
height = 480 | type: <class 'int'>
image_name = sample.png | type: <class 'str'>
is_color = True | type: <class 'bool'>
Total number of pixels: 307200
```

✓ **RESULT CHECK**

Total pixels = 307200 (expected 307200).

EXERCISE 2 if, elif, and else

An intensity below 0 or above 255 is labelled Invalid; otherwise 128 or above is a Bright pixel and anything lower is a Dark pixel. Six boundary values are tested.

basics.py · Exercise 2

```
# Exercise 2: if, elif, and else
print("\n=== Exercise 2: if, elif, and else ===")
test_values = [-1, 0, 127, 128, 255, 256]

for intensity in test_values:
    if intensity < 0 or intensity > 255:
        label = "Invalid"
    elif intensity >= 128:
        label = "Bright pixel"
    else:
        label = "Dark pixel"
    print(intensity, "->", label)
```

Output

```
=== Exercise 2: if, elif, and else ===
-1 -> Invalid
0 -> Dark pixel
127 -> Dark pixel
128 -> Bright pixel
255 -> Bright pixel
256 -> Invalid
```

✓ RESULT CHECK

Labels: Invalid, Dark, Dark, Bright, Bright, Invalid (matches expected).

EXERCISE 3 for and while Loops

A for loop visits [20, 130, 255] and labels each value using the threshold 128. A while loop prints 0, 1 and 2 and stops before 3 because the counter is updated on every pass.

basics.py · Exercise 3

```
# Exercise 3: for and while Loops
print("\n=== Exercise 3: for and while Loops ===")

# for loop: visit each pixel value and label it (threshold = 128)
for pixel in [20, 130, 255]:
    if pixel >= 128:
        print(pixel, "->", "Bright pixel")
    else:
        print(pixel, "->", "Dark pixel")

# while loop: print 0, 1, 2 and stop before 3
counter = 0
while counter < 3:
    print("counter =", counter)
    counter = counter + 1  # update the loop counter
```

Output

```
=== Exercise 3: for and while Loops ===
20 -> Dark pixel
130 -> Bright pixel
255 -> Bright pixel
counter = 0
counter = 1
counter = 2
```

✓ RESULT CHECK

for loop labels and while loop 0, 1, 2 are correct.

EXERCISE 4 Functions

is_bright(pixel, threshold=128) returns True when the pixel is at least the threshold, otherwise False. The default threshold can be overridden by the second argument.

basics.py · Exercise 4

```python
# Exercise 4: Functions
print("\n=== Exercise 4: Functions ===")


def is_bright(pixel, threshold=128):
    if pixel >= threshold:
        return True
    return False


print("is_bright(127) =", is_bright(127))        # Expected: False
print("is_bright(128) =", is_bright(128))        # Expected: True
print("is_bright(200, 210) =", is_bright(200, 210))  # Expected: False
```

Output

```
=== Exercise 4: Functions ===
is_bright(127) = False
is_bright(128) = True
is_bright(200, 210) = False
```

✓ RESULT CHECK

Results: False, True, False (matches expected).

EXERCISE 5 Lists and Tuples

The middle list item is replaced with 150, then 64 is appended. The tuple (640, 480) is unpacked into width and height.

basics.py · Exercise 5

```
# Exercise 5: Lists and Tuples
print("\n=== Exercise 5: Lists and Tuples ===")
pixels = [0, 128, 255]
print("First item:", pixels[0])

pixels[1] = 150      # replace the middle item
pixels.append(64)    # add a new item at the end
print("List:", pixels)           # Expected: [0, 150, 255, 64]
print("Length:", len(pixels))    # Expected: 4

size = (640, 480)
width, height = size  # tuple unpacking
print("width =", width)
print("height =", height)

# Why can a list item be changed but a tuple item cannot?
# A list is mutable: it is stored so its elements can be modified, added or
# removed in place (e.g. pixels[1] = 150). A tuple is immutable: once created,
# its elements cannot be reassigned, so size[0] = 100 would raise a TypeError.
# Tuples are therefore safe for fixed values such as an image's (width, height).
```

Output

```
=== Exercise 5: Lists and Tuples ===
First item: 0
List: [0, 150, 255, 64]
Length: 4
width = 640
height = 480
```

✓ RESULT CHECK

List = [0, 150, 255, 64], length = 4 (matches expected).

WHY CAN A LIST ITEM CHANGE BUT A TUPLE ITEM CANNOT?

A list is mutable: its items can be modified, added or removed in place (for example pixels[1] = 150). A tuple is immutable: once created its items cannot be reassigned, so size[0] = 100 would raise a TypeError. Tuples suit fixed values such as an image’s (width, height).

EXERCISE 6 Dictionaries and Sets

The key "mode" is added to the dictionary and values are read by key. Converting the label list to a set removes the duplicate "cat".

basics.py · Exercise 6

```
# Exercise 6: Dictionaries and Sets
print("\n=== Exercise 6: Dictionaries and Sets ===")
image = {"width": 640, "height": 480}
image["mode"] = "RGB"  # add a new key
print("Width:", image["width"])
print("Mode:", image["mode"])

labels = ["cat", "dog", "cat", "bird"]
unique_labels = set(labels)  # a set removes duplicates
print("Unique labels:", unique_labels)
print("Number of unique labels:", len(unique_labels))  # Expected: 3
```

Output

```
=== Exercise 6: Dictionaries and Sets ===
Width: 640
Mode: RGB
Unique labels: {'bird', 'dog', 'cat'}
Number of unique labels: 3
```

✓ RESULT CHECK

3 unique labels (set order may vary).

```
=== Exercise 1: Variables and Data Types ===
width = 640 | type: <class 'int'>
height = 480 | type: <class 'int'>
image_name = sample.png | type: <class 'str'>
is_color = True | type: <class 'bool'>
Total number of pixels: 307200
```

```
=== Exercise 2: if, elif, and else ===
-1 -> Invalid
0 -> Dark pixel
127 -> Dark pixel
128 -> Bright pixel
255 -> Bright pixel
256 -> Invalid
```

```
=== Exercise 3: for and while Loops ===
20 -> Dark pixel
130 -> Bright pixel
255 -> Bright pixel
counter = 0
counter = 1
counter = 2
```

```
=== Exercise 4: Functions ===
is_bright(127) = False
is_bright(128) = True
is_bright(200, 210) = False
```

```
=== Exercise 5: Lists and Tuples ===
First item: 0
List: [0, 150, 255, 64]
Length: 4
width = 640
height = 480
```

```
=== Exercise 6: Dictionaries and Sets ===
Width: 640
Mode: RGB
Unique labels: {'bird', 'cat', 'dog'}
Number of unique labels: 3
```

Figure 4 · basics.py code and output (Exercises 1–6)

6 STEP 6 Install the Required Packages

The three packages were installed with pip and then confirmed with pip show.

Command Prompt

```
dip-lab-01>python -m pip install numpy pillow matplotlib
dip-lab-01>python -m pip show numpy pillow matplotlib
```

NumPy
Numerical arrays

Pillow
Image tools

Matplotlib
Plotting

```
C:\Users\dell>python -m pip install numpy pillow matplotlib
Requirement already satisfied: numpy in c:\users\dell\appdata\local\python\pythoncore-3.14-64\lib\site-packages (2.5.3)
Requirement already satisfied: pillow in c:\users\dell\appdata\local\python\pythoncore-3.14-64\lib\site-packages (12.3.0)
Requirement already satisfied: matplotlib in c:\users\dell\appdata\local\python\pythoncore-3.14-64\lib\site-packages (3.11.2)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\dell\appdata\local\python\pythoncore-3.14-64\lib\site-packages (from matplotlib) (1.4.0)
Requirement already satisfied: cycler>=0.10 in c:\users\dell\appdata\local\python\pythoncore-3.14-64\lib\site-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.28.2 in c:\users\dell\appdata\local\python\pythoncore-3.14-64\lib\site-packages (from matplotlib) (4.65.0)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\dell\appdata\local\python\pythoncore-3.14-64\lib\site-packages (from matplotlib) (1.5.1)
Requirement already satisfied: packaging>=20.0 in c:\users\dell\appdata\local\python\pythoncore-3.14-64\lib\site-packages (from matplotlib) (26.3)
Requirement already satisfied: pyparsing>=3 in c:\users\dell\appdata\local\python\pythoncore-3.14-64\lib\site-packages (from matplotlib) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in c:\users\dell\appdata\local\python\pythoncore-3.14-64\lib\site-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in c:\users\dell\appdata\local\python\pythoncore-3.14-64\lib\site-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)
```

Figure 5 · Package installation

```
C:\Users\dell>python -m pip show numpy pillow matplotlib
Name: numpy
Version: 2.5.3
Summary: Fundamental package for array computing in Python
Home-page: https://numpy.org
Author: Travis E. Oliphant et al.
Author-email:
License-Expression: BSD-3-Clause AND 0BSD AND MIT AND Zlib AND CC0-1.0
Location: C:\Users\dell\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages
Requires:
Required-by: contourpy, matplotlib
---
Name: pillow
Version: 12.3.0
Summary: Python Imaging Library (fork)
Home-page: https://python-pillow.github.io
Author:
Author-email: Jeffrey 'Alex' Clark <aclark@aclark.net>
License-Expression: MIT-CMU
Location: C:\Users\dell\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages
Requires:
Required-by: matplotlib, reportlab
---
Name: matplotlib
Version: 3.11.2
Summary: Python plotting package
Home-page: https://matplotlib.org
Author: John D. Hunter, Michael Droettboom
Author-email: Unknown <matplotlib-users@python.org>
License: License agreement for matplotlib versions 1.3.0 and later
 ============================================================
```

Figure 6 · Installed package details

8 STEP 8 Write and Verify Import Statements

lab01_imports.py contains only the three import statements. It was run from the same project folder.

lab01_imports.py

```python
import numpy as np
from PIL import Image
import matplotlib.pyplot as plt
```

Command Prompt

```
dip-lab-01>python lab01_imports.py

dip-lab-01>
```

✓ RESULT CHECK

The program finished without errors and returned to the prompt. No output is expected because the file contains only imports.

D: > BS_SE_2024_28 > Semester 05 > Digital Image Processing > Lab > Tasks > DIP_Lab_01 > lab01_imports.py

```
1  import numpy as np
2  from PIL import Image
3  import matplotlib.pyplot as plt
4
```

Problems 3 Output Debug Console Terminal Ports Filter (e.g. text, !excludeText, t...

```
[Done] exited with code=0 in 0.217 seconds

[Running] python -u "d:\BS_SE_2024_28\Semester 05\Digital Image Processing\Lab\Tasks\DIP_Lab_01\lab01_imports.py"

[Done] exited with code=0 in 2.904 seconds
```

Figure 7 · Successful import run

SUMMARY

Python vs pip, and the Purpose of Imports

Python

The programming language and interpreter that reads and runs our .py files.

pip

Python's package manager. It downloads and installs extra libraries such as NumPy, Pillow and Matplotlib.

Running pip as "python -m pip" guarantees that packages are installed for the same Python that runs our code. The import statements then load these installed libraries into a program so their features can be used.

Statement	Purpose
`import numpy as np`	Loads NumPy (shortened to np) for fast numerical arrays
`from PIL import Image`	Loads Pillow's Image class for opening and handling images
`import matplotlib.pyplot as plt`	Loads Matplotlib's plotting module (shortened to plt) for graphs and image display

CONCLUSION

Python and VS Code were set up successfully, the core Python concepts (variables, conditions, loops, functions, lists, tuples, dictionaries and sets) were practised, and NumPy, Pillow and Matplotlib were installed and verified through error-free imports. The environment is now ready for the upcoming Digital Image Processing labs.

SUBMISSION CHECKLIST

Criterion	Where it is in this report	Marks
Installation and setup	Steps 1–4 (Figures 1–3)	**2**
Variables and conditions	Step 5, Exercises 1–2	**2**

Loops and functions	Step 5, Exercises 3–4	**2**
Collections	Step 5, Exercises 5–6 (with mutability explanation)	**3**
Packages and imports	Steps 6 and 8	**2**
Report and understanding	Details on cover, Python vs pip section, files below	**1**
Files submitted with report	hello.py · basics.py · lab01_imports.py	**12 total**